A proposal for a type trait to detect narrowing conversions

ISO/IEC JTC1 SC22 WG21 P0870R1 2017-11-21

Project: Programming Language C++

Audience: Library Evolution Working Group Incubator

Giuseppe D'Angelo, giuseppe.dangelo@kdab.com

Introduction

This paper proposes a new type trait for the C++ Standard Library, `is_narrowing_convertible`, to detect whether a type is implictly convertible to another type by going through a narrowing conversion.

Changelog

- P0870R1: submitted for the Prague mailing, targeting LEWGI.

Motivation and Scope

A *narrowing conversion* is formally defined by the C++ Standard in [dcl.init.list], with the intent of forbidding them from list-initializations.

It is useful in certain contexts to know whether a conversion between two types would qualify as a narrowing conversion. For instance, it may be useful to inhibit (via SFINAE) construction or conversion of "wrapper" types (like `std::variant` or `std::optional`) when a narrowing conversion is requested.

A use case the author has recently worked on was to prohibit narrowing conversions when establishing a connection in the Qt framework (see [Qt]), with the intent of making the type system more robust. Simplifying, a connection is established between a *signal* non-static member function of an object and a *slot* non-static member function of another object. The signal's and the slot's argument lists must have compatible arguments for the connection to be established: the signal must have at least as many arguments as the slot, and the each argument of the signal must be implictly convertible to the corresponding argument of the slot. In case of a mismatch, a compile-time error is generated. Bugs have been observed when connections were successfully established, but a narrowing conversion (and subsequent loss of data/precision) was happening. Detecting such narrowing conversions, and making the connection fail for such cases, would have prevented the bugs.

Impact On The Standard

This proposal is a pure library extension.

It proposes changes to an existing header, `<type_traits>`, but it does not require changes to any standard classes or functions and it does not require changes to any of the standard requirement tables.

This proposal does not require any changes in the core language, and it has been implemented in standard C++.

This proposal does not depend on any other library extensions.

This proposal affects [P0608R0], a proposed resolution for [LEWG 227]. The type trait described by this proposal has similar semantics to the exposition only `is_non_narrowing_convertible_v` trait defined by [P0608R0]. The semantics described by [P0608R0] are however different from the ones defined by this proposal: in particular, the present proposal does not address boolean conversions [conv.bool], and the present proposal uses a positive tense instead of a negative one.

Design Decisions

The most natural place to add the trait presented by this proposal is the already existing `<type_traits>` header, which collects most (if not all) of the type traits available from the Standard Library.

In the `<type_traits>` header the `is_convertible` type trait checks whether a type is implictly convertible to another type. Building upon this established name, the proposed name for the trait described by this proposal shall therefore be `is_narrowing_convertible`, with the corresponding `is_narrowing_convertible_v` type trait helper variable template.

Should `is_convertible_v<From, To> == true` be a precondition of trying to instantiate `is_narrowing_convertible<From, To>`?

The author deems this unnecessary. Adding such a precondition would likely make the usage of the proposed trait more difficult and/or error prone.

First and foremost, it's not going to simplify the specification for the proposed trait, or its implementation (as far as the author can see). Second, the current wording for [dcl.init.list] restricts narrowing conversion to a fixed number of cases: between certain integer and floating point types, as well as from unscoped enumeration types to a integer or a floating point type. Any other conversion is never a narrowing conversion; therefore, there is no need of requiring `is_convertible_v` to be `true`.

`is_convertible`'s own prerequisites are another discussion point (they would become `is_narrowing_convertible`'s prerequisites). In [meta.rel], the *Type relationship predicates* table says that the prerequisites for `is_convertible<From, To>` are:

> `From` and `To` shall be complete types, arrays of unknown bound, or *cv* void types.

These prerequisites are not necessary for `is_narrowing_convertible`: for instance, none of the types that can be involved in a narrowing conversions can possibly be incomplete.

Should `is_narrowing_convertible_v<From, To>` yield `false` for boolean conversions [conv.bool]?

The author deems this to be out of scope for the proposed trait, which should have only the semantics defined by the language regarding narrowing conversion. Another trait should be therefore added to detect (implicit) boolean conversions, which, according to the current wording of [dcl.init.list], are never considered narrowing conversions.

It is the author's opinion that, in general, implicit boolean conversions are undesiderable for the same reasons that make narrowing conversions unwanted in certain contexts — in the sense that a conversion towards bool may discard important information. However, we recognize the importance of not imbuing a type trait with extra, not yet well-defined semantics, and therefore we are excluding boolean conversions from the present proposal.

Bikeshedding: `is_narrowing_convertible` or `is_non_narrowing_convertible`?

The Standard Library does not currently have type traits with a negation in their names, so the current proposal uses a positive tense, although we foresee that the majority of the usages are going to be about detecting whether there is *no* narrowing conversion between two given types. Note that Standard Library names such as `is_nothrow_constructible` are not considered to be negations; the tense is positive, using `nothrow` as an adjective/attribute.

Bikeshedding: `is_narrowing_convertible` or `is_narrowing`?

This paper proposes the former, building on top of the existing `is_convertible` name; however, we concede that `is_narrowing` could lead to cleaner code. Moreover, at the present time, there should be no ambiguity over the meaning of `is_narrowing`: the only usage of "narrowing" in the Standard is about narrowing conversions. Nonetheless, it may make sense to future-proof the name proposed here, and therefore go for the slightly more verbose `is_narrowing_convertible`.

Technical Specifications

The proposed type trait has already been implemented in various codebases using ad-hoc solutions (depending on the quality of the compiler, etc.). The most straighforward implementations rely on SFINAE using an expression such as `To{std::declval<From>()}`.

Standardese

In [meta.type.synop], add to the the header synopsis:

```
  template <class From, class To> struct is_convertible;
  template <class From, class To> struct is_narrowing_convertible;
```

```
  template <class From, class To>
    inline constexpr bool is_convertible_v = is_convertible<From, To>::value;
  template <class From, class To>
    inline constexpr bool is_narrowing_convertible_v = is_narrowing_convertible<From, To>::value;
```

In [meta.rel], add a new row to the *Type relationship predicates* table, after the entry for `is_convertible`:

> *Template* `template <class From, class To> struct is_narrowing_convertible;`
>
> *Condition* *see below*
>
> *Comments* —

Modify paragraph 5 of [meta.rel]:

> 5. The predicate condition for a template specialization `is_convertible<From, To>` or a template specialization `is_narrowing_convertible<From, To>` shall be satisfied if and only if the return expression in the following code would be well-formed, including any implicit conversions to the return type of the function; for `is_narrowing_convertible<From, To>` the conversion shall moreover be a narrowing conversion ([dcl.init.list]):
>
> ```
> To test() {
>   return declval<From>();
> }
> ```
>
> [*Note:* This requirement gives well-defined results for reference types, void types, array types, and function types. — *end note*] [*Note:* For the purpose of establishing whether the conversion is a narrowing conversion, the source is never a constant expression. — *end note*]

Acknowledgements

Thanks to KDAB for supporting this work.

Thanks to Marc Mutz, André Somers and Zhihao Yuan for their feedback.

References

[Qt] Qt version 5.9, *QT_NO_NARROWING_CONVERSIONS_IN_CONNECT*, QObject class documentation

[P0608R0] Zhihao Yuan, *A sane variant converting constructor (LEWG 227)*

[LEWG 227] Zhihao Yuan, *Bug 227 - variant converting constructor allows unintended conversions*